

Avanceret SQL og MongoDB queries

Nøgleord: SQL • MongoDB • Avanceret queries • windows functions • stored procedures

Ressourcer: [PostgreSQL Window Functions Docs](#) [MongoDB \\$setWindowFields](#) [PostgreSQL CREATE FUNCTION](#)

Session 9: Avanceret-SQL og MongoDB

I denne session vil vi dykke ned i forskellige aggregations funktioner for MongoDB og SQL, og dykke ned i mere avanceret database koncepter som Windows funktions, CTE's, og stored procedures

Aggregeringsfunktioner, revisited

Til disse opgaver skal i arbejde 2 og 2

Opgave 1

Gå ind i Kandidaportalens kodebase og kig i `admin.tsx` routen. Her er flere eksempler på aggregeringsfunktioner i MongoDB, bl.a.:

- Linje 104-147, her er en pipeline der ender i variabelen `jobPreferencesByEducation`
- Linje 196-218, her er en pipeline der ender i variabelen `educationStats`
- Linje 238-267, her er en pipeline der ender i variabelen `contactedEducations`

I hver af de filer er der flere avanceret aggregerings-funktioner skrevet i MongoDB.

- Vælg en af disse datapipelines som jeres fokusområde. Hvis i er i tvivl så vælg den korteste, `educationStats`
- I skal nu først forstå hvad pipelinen konkret gør. Skriv hvad hvert skridt gør ned i normalt sprog ned i en step by step forklaring, som skal se sådan her ud:
 - Skridt et: Henter data fra collectionen `applications`
 - Skridt to: Filtrerer data, så vi kun har data med `education` feltet sat til `true`
 - Skridt tre...
- Når i har forstået hvad pipelinen gør, så skal i prøve at omskrive den til SQL. Lige mærke til forskelle mellem MongoDB og SQL
- BONUS: Hvis i er hurtigt færdige, så prøv at omskrive en af de andre pipelines i `admin.tsx` til SQL også, eller find en større og mere kompleks pipeline i kodebasen og omskriv den til SQL

Window Functions & views

I disse opgaver skal i arbejde med at lave windows functions og views i en online SQL editor. Der er et link på dagens lektion der fører til en online editor med en test database, som i kan bruge til at lave jeres queries i.

Opgave 2

- Kig på den database i har fået stillet til rådighed og se hvad relationerne er i databasen. Prøv at forstå hvad de forskellige tabeller indeholder, og hvordan de relaterer til hinanden.
- Lav en window funktion, der gør følgende:

i) Rangere hver post baseret på deres oprettelsesdato, så den nyeste post får rang 1

ii)

iii) Rangere hver post baseret på antallet af comments (kræver at i joiner mellem posts og comments tabellerne)

iv) Rangere hver post baseret på antallet af likes

v) Tildel en rangering på hver post der hedder `number_post_for_author`, der siger hvilket nummer post det er for den pågældende forfatter

c) Læs op på hvordan window functions og views fungerer i MongoDB på følgende links:

- <https://www.mongodb.com/docs/manual/reference/operator/aggregation/setWindowFields/#std-label-setWindowFields-window-operators>

Vælg en af jeres løsninger på en forrige opgave og omskriv den til en MongoDB aggregerings pipeline, der bruger `$setWindowFields` og relevante window operators til at opnå det samme resultat som jeres SQL query gjorde.